VIVA GUIDE 02 / 05

The application API

Express routes, conversation sessions, persistence, validation, and service-to-service requests.

YOUR RESPONSIBILITY

Own the Node.js boundary between the browser, saved conversation data, and the AI service.

THE SHORT VERSION

The Express server exposes the browser-facing API. It validates the question, finds or creates a JSON-backed session, saves the user turn, calls FastAPI, saves the assistant turn and source list, and returns a stable response shape to the client.

A 30-SECOND OPENING

My part is the Express API. It is the integration boundary: it manages conversations, persists their messages, and forwards a question to the AI service without putting Gemini credentials in the frontend.

IN THIS GUIDE

- What the backend owns
- Session data and persistence
- Chat route, step by step
- Other routes and important boundaries
- Error handling and a useful incident explanation
- Demo and viva questions

WHAT THE BACKEND OWNS

The Node/Express service listens on the configured port (5000 by default). Its route and middleware setup is in server/src/server.js; settings are in config.js; JSON persistence helpers are in dataStore.js. Axios is used for calls to FastAPI.

- Conversation lifecycle: create, list, fetch, rename, and delete sessions.
- Chat orchestration: validate the message, persist both turns, call the AI service, and return answer plus sources.
- Health aggregation: report Express status and proxy a small amount of AI readiness information.
- Demo support: feedback echo and document-upload metadata routes.

This is an academic local build. Its JSON persistence is not MongoDB, and the health response explicitly reports that Mongo is not configured.

SESSION DATA AND PERSISTENCE

A session has an id, title, createdAt, updatedAt, and messages array. A message has an id, sessionId, role, content, timestamp, and optionally sources/error metadata. UUIDs provide IDs. dataStore.js reads and writes local JSON files under server/src data storage.

The session routes use a read-modify-save pattern. A missing session lookup returns 404 for fetch, patch, or delete. For chat, a missing or unknown session id causes the server to create a fresh session so a question can still be accepted.

- POST /api/sessions creates an empty conversation and returns 201.
- GET /api/sessions lists stored conversations; GET /api/sessions/:id loads one.
- PATCH /api/sessions/:id updates a title; DELETE /api/sessions/:id removes its data.
- The file-based store is simple to inspect and reset during a demo, but is not a concurrency-safe production database.

CHAT ROUTE, STEP BY STEP

```
POST /api/chat
1. Read sessionId and message; trim message.
2. Return 400 if the question is blank.
3. Find or create the session.
4. Append and persist the USER message.
5. POST question + latest 10 turns to FastAPI.
6. Persist the ASSISTANT answer and source metadata.
7. Return sessionId, messageId, answer, sources, timestamp.
```

The AI request has a 20-second timeout. If the AI request fails, Express stores a user-friendly assistant fallback with an error flag and returns it in the normal response envelope. This keeps the chat UI from receiving an unhandled server exception, but it also means a provider failure is not always reflected as an HTTP error to the browser.

A global JSON body limit of 2 MB, CORS middleware, and a rate limit of 120 requests per minute are configured. Blank questions are rejected before creating a stored user turn. The server persists the user turn before the AI call so the original question remains in the history even when generation fails.

OTHER ROUTES AND IMPORTANT BOUNDARIES

GET /api/health calls the FastAPI health endpoint with a short timeout. If FastAPI is unavailable, the response can

still identify Express as running while reporting the AI service as unavailable. This is a dependency-aware health check, not proof that every downstream request will succeed.

POST /api/feedback validates messageId and rating but currently echoes the data; it does not persist feedback. Admin document routes accept PDF/TXT uploads, store upload metadata, and call the AI indexing endpoint. In the current implementation, that endpoint normalizes supplied text but does not parse the uploaded file or update the active official NIT index.

- Do not tell a viva panel that admin uploads dynamically replace or expand the NIT knowledge base.
- The official indexed-document workflow is scripts/ingest_nith_documents.py, not the admin upload route.
- Admin routes do not show authentication/authorization middleware in this build.

ERROR HANDLING AND A USEFUL INCIDENT EXPLANATION

A 502 from FastAPI can be caused by Gemini returning an error or the AI service mapping a generation failure to HTTP 502. A missing Gemini key with a retrieved match is reported as 503 by FastAPI. Express catches downstream failures and can store a fallback answer. Check the AI health response for key/index readiness, then inspect the AI service logs and model/key configuration.

- Client-to-Express failure: the browser displays its service-unavailable message.
- Express-to-FastAPI failure: Express stores and returns the AI-unavailable fallback.
- FastAPI-to-Gemini failure: FastAPI reports a generation error; Express may convert it to that fallback.
- No retrieved passage: FastAPI returns a grounded not-found answer without calling Gemini.

DEMO AND VIVA QUESTIONS

- Start the backend and AI service, check /api/health, then create a session in the browser.
- Send a question and inspect its stored USER and ASSISTANT messages plus returned sources.
- Disconnect or misconfigure the AI service and explain the saved fallback behavior.
- Point to dataStore.js to explain why this persistence is suitable for a local demonstration but not multiple production workers.

Why use a backend between frontend and AI service?
It centralizes session storage and orchestration, and protects provider credentials from browser exposure.

Why save the user question before calling AI?
It preserves the submitted turn if the downstream service fails and makes the conversation auditable locally.

What is the persistence limitation?
JSON files are process-local/simple storage; concurrent writes, scaling, backups, and access control need a database design.

Why can the chat endpoint return HTTP 200 when AI failed?
The route returns a user-friendly fallback as a normal chat result; this is a UX choice but can hide failure from HTTP-level monitoring.

Does document upload update retrieval?
Not in the current active knowledge-base flow. The upload route stores metadata and its index route only acknowledges normalized text.

What does the rate limiter do?

It caps requests per IP window at 120 per minute to reduce accidental or abusive bursts; it is not user authentication.